

Open in app ↗

Sign up

Sign in

Medium

 Search

Building an AWS Enterprise Landing Zone with Terraform: A Complete Guide

10 min read · Jan 30, 2026



Ravi Yasakeerthi

Follow



Listen



Share

A practical approach to setting up a secure, scalable multi-account AWS infrastructure

Introduction

When organizations grow, managing AWS resources in a single account becomes challenging. Security risks increase, cost tracking becomes difficult, and teams step on each other's toes. The solution? A well-designed **Landing Zone** - a multi-account AWS environment with proper governance, security, and access management.

In this article, I'll walk you through building an enterprise-grade AWS Landing Zone using Terraform. This is a real implementation that I built for a portfolio project, simulating a mid-size organization with around 500 employees.

Here is what we are going to discuss in this post:

- Setting up AWS Organizations with a multi-account strategy
- Designing Organizational Units (OUs) for different teams and environments
- Using Service Control Policies (SCPs) as security guardrails
- Configuring IAM Identity Center (SSO) for centralized access management
- Enabling security services like CloudTrail and GuardDuty across all accounts
- Structuring Terraform modules so the code stays clean and reusable

Why Multi-Account Strategy?

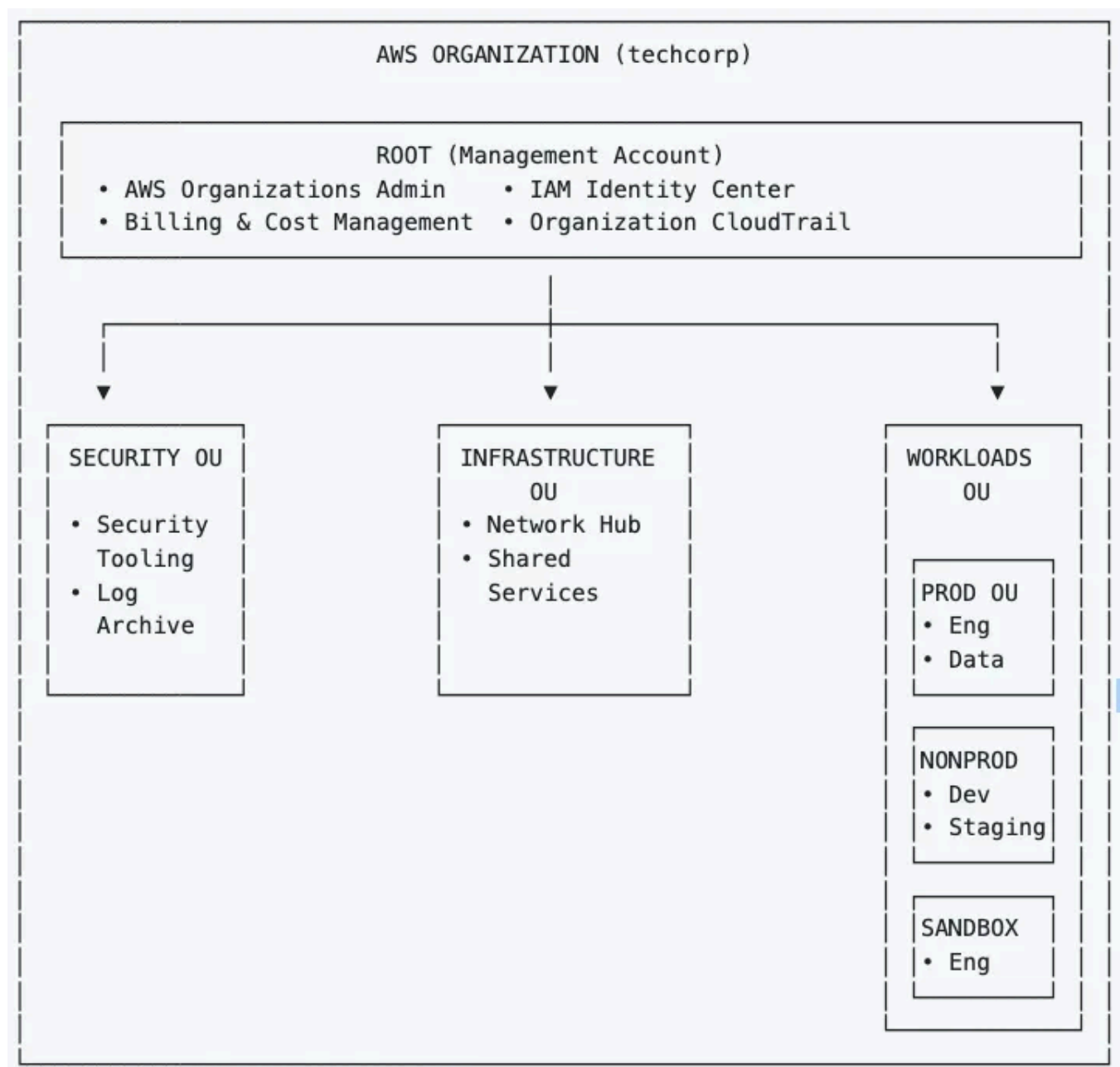
Before diving into the implementation, let me explain why we need multiple AWS accounts in the first place.

With a single AWS account, one misconfiguration can affect everything. Cost tracking becomes hard because all teams share the same billing. There are no real security boundaries between environments, and IAM policies become unmanageable as the team grows. On top of that, all teams share the same service quotas.

A multi-account setup solves all of these problems. Production is separated from development. If one account gets compromised, others remain safe. Each account has its own billing, making cost tracking straightforward. Compliance becomes easier because you can enforce rules per account. And teams get the autonomy to manage their own accounts without affecting others.

Architecture Overview

Here is the high-level architecture of our Landing Zone:



Account Structure

```
Root (Management Account)
├── Security OU
│   ├── security-tooling (GuardDuty, Security Hub admin)
│   └── log-archive (Centralized logs)
├── Infrastructure OU
│   ├── network-hub (Transit Gateway, DNS)
│   └── shared-services (CI/CD, Container Registry)
├── Workloads OU
│   ├── Production OU
│   │   ├── prod-engineering
│   │   └── prod-data
│   ├── Staging OU
│   │   ├── dev-engineering
│   │   └── staging-engineering
│   └── Experimental OU
│       └── sandbox-engineering
└── Suspended OU (for decommissioned accounts)
```

Design Decisions

Why this structure?

The Security OU sits at the root level because security accounts need organization-wide visibility. Infrastructure has its own OU because shared services should not mix with workloads. The Workloads OU has nested OUs so we can apply different policies for production versus non-production. And the Suspended OU gives us a safe place to move accounts before closing them.

Repository Structure

A well-organized repository is crucial for maintainability. Here is the structure I used:

```
aws_devops/
├── terraform/
│   ├── environments/
│   ├── management/
│   │   ├── main.tf
│   │   └── terraform.tfvars.example
│   └── modules/
```

```
organization/  
  main.tf  
  scps.tf  
  variables.tf  
  outputs.tf  
identity-center/  
  main.tf  
  assignments.tf  
  users.tf  
  variables.tf  
  outputs.tf  
security/  
  cloudtrail/  
  guardduty/
```

The environments/ directory holds account-specific configurations. The modules/ directory contains reusable Terraform modules. Modules should be reusable and environment-agnostic. Environment directories contain only the specific configuration for each account.

Implementation Deep Dive

Step 1: AWS Organizations Setup

The foundation of our Landing Zone is AWS Organizations. It allows centralized management of multiple accounts.

```
resource "aws_organizations_organization" "this" {  
  aws_service_access_principals = [  
    "access-analyzer.amazonaws.com",  
    "cloudtrail.amazonaws.com",  
    "config.amazonaws.com",  
    "guardduty.amazonaws.com",  
    "sso.amazonaws.com",  
  ]  
  
  enabled_policy_types = [  
    "SERVICE_CONTROL_POLICY",  
    "TAG_POLICY",  
    "BACKUP_POLICY",  
  ]  
  
  feature_set = "ALL"  
}
```

A few important points here. The `aws_service_access_principals` list defines which AWS services can operate across your organization. The `enabled_policy_types` controls what types of policies you can attach to OUs. And `feature_set = "ALL"` enables all organization features, which is required for SCPs.

Step 2: Organizational Units

OUs are containers for accounts. They help you apply policies to groups of accounts.

```
resource "aws_organizations_organizational_unit" "security" {
  name      = "Security"
  parent_id = aws_organizations_organization.this.roots[0].id

  tags = {
    Description = "Security and compliance accounts"
    ManagedBy   = "terraform"
  }
}

resource "aws_organizations_organizational_unit" "workloads" {
  name      = "Workloads"
  parent_id = aws_organizations_organization.this.roots[0].id
}

resource "aws_organizations_organizational_unit" "production" {
  name      = "Production"
  parent_id = aws_organizations_organizational_unit.workloads.id
}
```

Organization Actions ▾

Organizational units (OUs) enable you to group several accounts together and administer them as a single unit instead of one at a time.

Search by name, email, account ID or OU ID. Hierarchy | List

Organizational structure | Account created/joined date

- ▼ **Root** (ou-9zgk)
 - ▼ **Infrastructure** (ou-9zgk-y4vkm5a9)
 - ▼ **Security** (ou-9zgk-hipne5rd)
 - techcorp-log-archive (697, s-logs@gmail.com) Created 2026/01/25
 - techcorp-security-tooling (616, rity@gmail.com) Created 2026/01/25
 - ▼ **Suspended** (ou-9zgk-6goleix3)

This resource is empty
 - ▼ **Workloads** (ou-9zgk-fu29oa8t)
 - ▼ **Experimental** (ou-9zgk-rejd0v7n)
 - techcorp-sandbox-engineering (3071, fbox@gmail.com) Created 2026/01/25
 - ▼ **Production** (ou-9zgk-3bbqqhb0)
 - techcorp-prod-data (6720, l-data@gmail.com) Created 2026/01/25
 - techcorp-prod-engineering (383, id-eng@gmail.com) Created 2026/01/25
 - ▼ **Staging** (ou-9zgk-p84dy8in)
 - techcorp-dev-engineering (576, v-eng@gmail.com) Created 2026/01/25
 - techcorp-staging-engineering (9050, jing@gmail.com) Created 2026/01/25
 - techcorp-demo (73, com) Joined 2026/01/25 (management account)

Step 3: Member Accounts

Creating accounts is straightforward, but there is a catch — AWS accounts cannot be deleted via Terraform. They must be manually closed through the AWS Console.

```
resource "aws_organizations_account" "security_tooling" {
  count = var.create_accounts ? 1 : 0
}
```

```

name      = "${var.organization_name}-security-tooling"
email     = var.account_emails["security_tooling"]
parent_id = aws_organizations_organizational_unit.security.id

iam_user_access_to_billing = "DENY"
role_name                  = "OrganizationAccountAccessRole"

lifecycle {
  ignore_changes = [role_name, iam_user_access_to_billing]
}
}

```

Notice the count variable. This lets you set up the organization structure first, then create accounts in a separate step. This is important because account creation is a one-way operation.

Email Address Strategy

Each AWS account needs a unique email address. For testing, I used Gmail's alias feature:

Base email: yourname@gmail.com

Generated aliases:

- yourname+aws-security@gmail.com
- yourname+aws-logs@gmail.com
- yourname+aws-network@gmail.com

All emails arrive in your single inbox. Here is how I configured this in Terraform:

```

locals {
  email_parts = split("@", var.base_email)
  email_user  = local.email_parts[0]
  email_domain = local.email_parts[1]

  generated_emails = {
    security_tooling = "${local.email_user}+aws-security@${local.email_domain}"
    log_archive      = "${local.email_user}+aws-logs@${local.email_domain}"
    network_hub      = "${local.email_user}+aws-network@${local.email_domain}"
  }
}

```

Step 4: Service Control Policies (SCPs)

SCPs are guardrails that limit what actions can be performed in member accounts. They are attached to OUs and apply to all accounts within that OU.

```
resource "aws_organizations_policy" "deny_root_user" {
  name          = "DenyRootUser"
  description    = "Deny all actions for root user in member accounts"
  type          = "SERVICE_CONTROL_POLICY"

  content = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "DenyRootUser"
        Effect   = "Deny"
        Action   = "*"
        Resource = "*"
        Condition = {
          StringLike = {
            "aws:PrincipalArn" = "arn:aws:iam::*:root"
          }
        }
      }
    ]
  })
}
```

Here are the SCPs I implemented:

- DenyRootUser — blocks root user actions, attached to Workloads OU
- DenyLeaveOrganization — prevents accounts from leaving, attached to Root
- RegionRestriction — allows only specific AWS regions, attached to Root
- DenyPublicS3 — blocks public S3 buckets, attached to Workloads OU
- ProtectSecurityServices — prevents disabling security tools, attached to Workloads and Security OUs
- SandboxRestrictions — limits expensive services, attached to Experimental OU

Service control policies

Service control policies (SCPs) offer central control over the maximum available permissions for IAM users and IAM roles in an organization. [Learn more](#)

[Disable service control policies](#)

Available policies [Actions](#) [Create policy](#)

☐	Name	Kind	Description
☐	DenyLeaveOrganization	Customer managed policy	Prevent accounts from leaving the organization
☐	DenyPublicS3Buckets	Customer managed policy	Prevent creation of public S3 buckets
☐	DenyRegionRestriction	Customer managed policy	Restrict deployments to allowed regions only (ap-southeast-2, ap-southeast-1)
☐	DenyRootUserActions	Customer managed policy	Deny all actions by the root user
☐	FullAWSAccess	AWS managed policy	Allows access to every operation
☐	ProtectSecurityServices	Customer managed policy	Prevent modification of security services (CloudTrail, GuardDuty, Config)
☐	RequireMDSv2	Customer managed policy	Require Instance Metadata Service Version 2 for EC2 instances
☐	SandboxRestrictions	Customer managed policy	Additional restrictions for sandbox/experimental accounts

DenyRootUserActions [Delete](#) [Edit policy](#)

Policy details

Name: DenyRootUserActions

ARN: arn:aws:organizations::policy:policy/p-hemghogk

Policy type: Service control policy (customer managed)

Description: Deny all actions by the root user

Content [Targets](#) [Tags](#)

```
{
  "Statement": [
    {
      "Action": "*",
      "Condition": {
        "StringLike": {
          "aws:PrincipalArn": "arn:aws:iam::*:root"
        }
      },
      "Effect": "Deny",
      "Resource": "*",
      "Sid": "DenyRootUserActions"
    }
  ],
  "Version": "2012-10-17"
}
```

Step 5: IAM Identity Center (SSO)

IAM Identity Center provides centralized access management. Users log in once and can access multiple accounts with different permission levels.

There are three main components:

1. Permission Sets — define what users can do (for example, AdministratorAccess or ReadOnly)
2. Groups — organize users by role
3. Account Assignments — connect groups to accounts with specific permission sets

```
resource "aws_ssoadmin_permission_set" "administrator" {
  name            = "AdministratorAccess"
  description     = "Full administrator access"
  instance_arn    = local.sso_instance_arn
  session_duration = "PT4H"
}

resource "aws_ssoadmin_managed_policy_attachment" "administrator" {
  instance_arn        = local.sso_instance_arn
  managed_policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
  permission_set_arn = aws_ssoadmin_permission_set.administrator.arn
}
```

Groups are created in the Identity Store and then assigned to accounts:

```

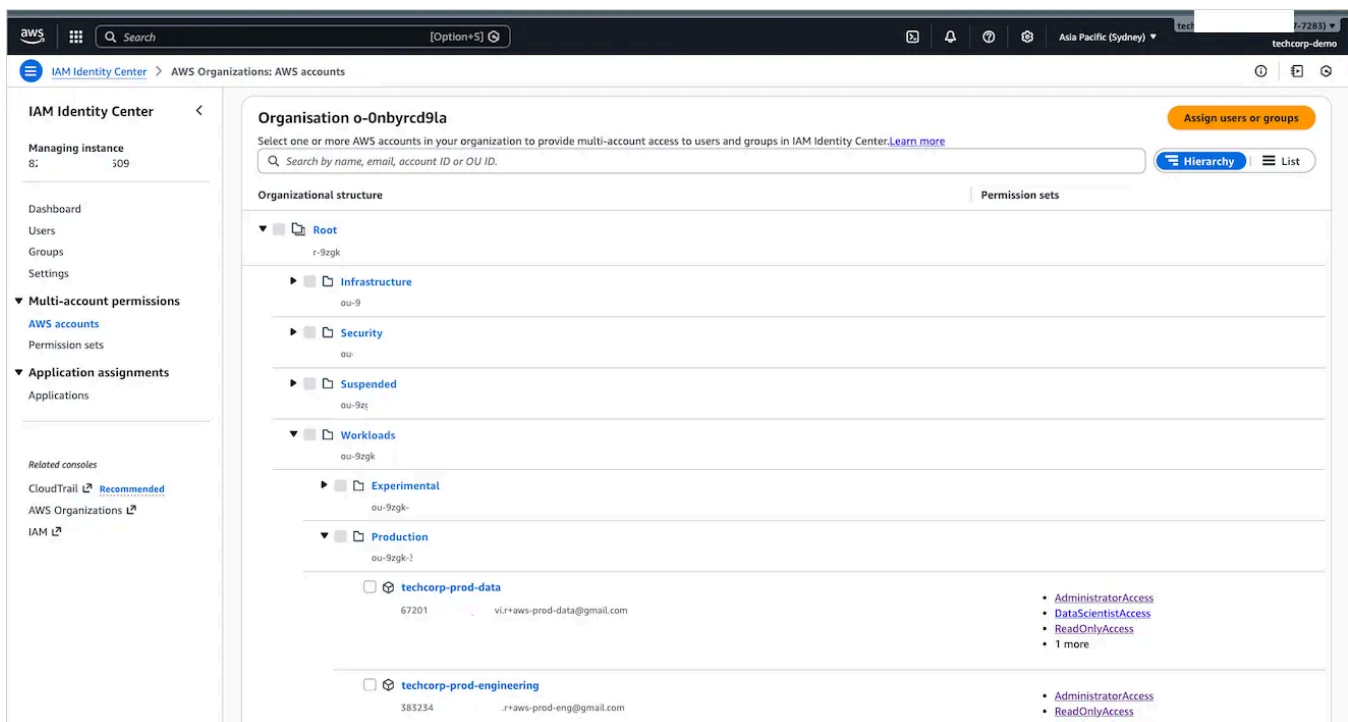
resource "aws_identitystore_group" "platform_admins" {
  display_name      = "Platform-Admins"
  description       = "Platform administrators with full access"
  identity_store_id = local.identity_store_id
}

resource "aws_ssoadmin_account_assignment" "platform_admins_management" {
  instance_arn      = local.sso_instance_arn
  permission_set_arn = aws_ssoadmin_permission_set.administrator.arn

  principal_id      = aws_identitystore_group.platform_admins.group_id
  principal_type     = "GROUP"

  target_id         = var.management_account_id
  target_type       = "AWS_ACCOUNT"
}

```



Access Matrix

Get Ravi Yasakeerthi's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

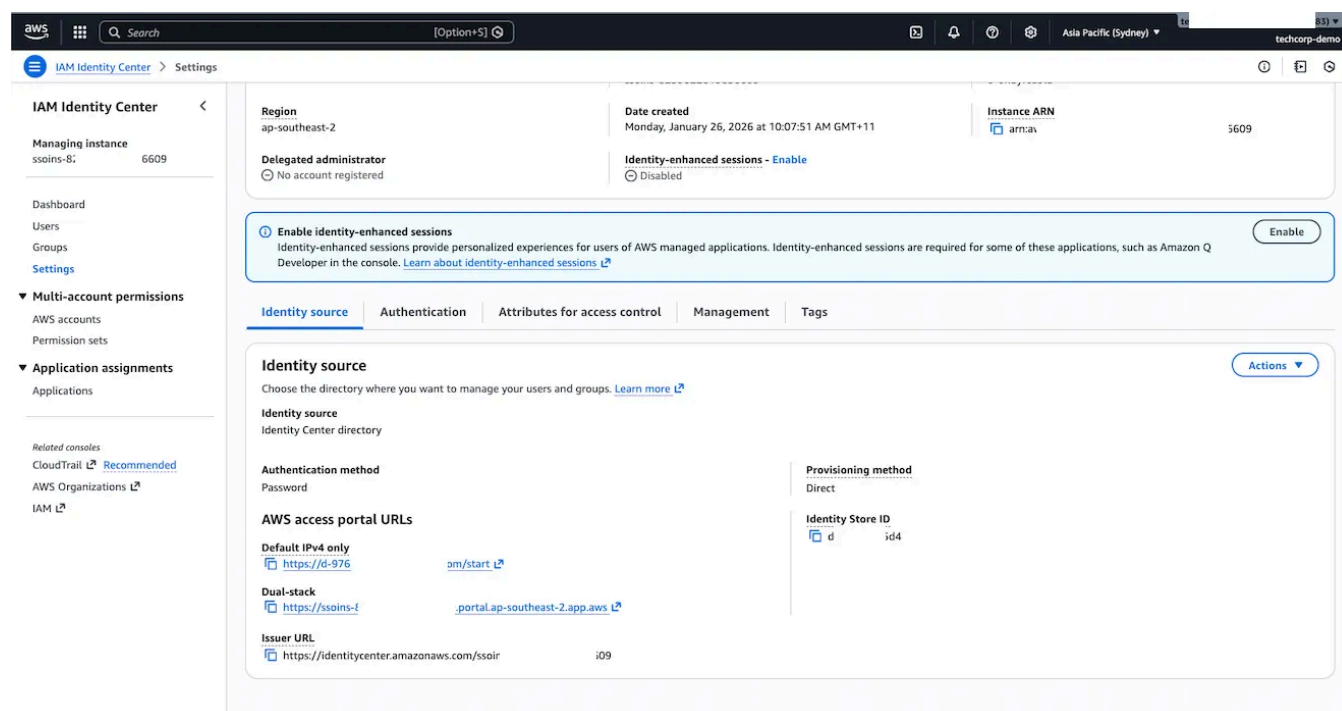
☐ Remember me for faster sign in

Here is how I mapped groups to accounts:

- Platform-Admins get Administrator access to all accounts
- Security-Team gets Administrator access to security accounts and SecurityAudit access to all accounts
- Engineering-Leads get PowerUser access to non-prod accounts and ReadOnly access to prod accounts
- Engineering-Developers get Developer access to dev and sandbox accounts
- DataScience-Team gets DataScientist access to the prod-data account
- Auditors get ReadOnly access to all accounts

Identity Source Options — What to Use in Production

In this project, I created demo users directly in IAM Identity Center for testing purposes. However, in a real production environment, you would not manage users this way. There are three identity source options available in AWS, and the right choice depends on your organization.



The first option is the IAM Identity Center built-in directory. This is what I used for this demo. It is the simplest to set up and works well for small teams, testing, and proof-of-concept projects. You create and manage users directly in the IAM Identity Center console or via Terraform.

```
resource "aws_identitystore_user" "demo" {  
  identity_store_id = local.identity_store_id  
  display_name      = "Platform Admin"  
  user_name         = "admin"  
  
  emails {  
    value      = "user+sso-admin@gmail.com"  
    primary    = true  
  }  
}
```

The second option is Active Directory. If your organization already uses Microsoft Active Directory, you can connect it to IAM Identity Center. This works with both on-premises AD (through AD Connector) and AWS Managed Microsoft AD. Users authenticate with their existing AD credentials, groups sync automatically, and password policies are managed centrally in AD.

On-premises AD → AD Connector → IAM Identity Center

or

AWS Managed Microsoft AD → IAM Identity Center

The third option is an External Identity Provider (IdP). This is for organizations already using identity providers like Okta, Azure AD (Entra ID), Google Workspace, or OneLogin. The integration works through SAML 2.0 for authentication and SCIM for automatic user provisioning. This means when you add or remove a user in Okta, the change automatically reflects in IAM Identity Center.

External IdP (Okta / Azure AD)

| — SAML 2.0 → IAM Identity Center (authentication)

| — SCIM → IAM Identity Center (user provisioning)

For most enterprises, I recommend the External Identity Provider approach because:

1. Single source of truth — manage users in one place
2. Automated provisioning — SCIM handles user creation and deletion
3. Consistent MFA — same multi-factor authentication across all applications
4. Compliance — easier audit trail for user access
5. Offboarding — disable a user in the IdP and access is revoked everywhere

The good news is that the Terraform configuration for groups, permission sets, and account assignments remains the same regardless of which identity source you choose. Only the user creation part changes.

Step 6: Security Services

CloudTrail

CloudTrail logs all API calls across your organization. This is essential for security auditing and compliance.

```
resource "aws_cloudtrail" "organization" {
  name                = "${var.name_prefix}-organization-trail"
  s3_bucket_name      = aws_s3_bucket.cloudtrail.id
  is_organization_trail = true
  is_multi_region_trail = true
  enable_log_file_validation = true
  kms_key_id          = aws_kms_key.cloudtrail.arn

  event_selector {
    read_write_type      = "All"
    include_management_events = true
  }
}
```

Setting `is_organization_trail = true` means this single trail captures API calls from all accounts in the organization. All logs are stored in a centralized S3 bucket with KMS encryption.

GuardDuty

GuardDuty is a threat detection service. It analyzes CloudTrail logs, VPC Flow Logs, and DNS logs to identify threats.

```

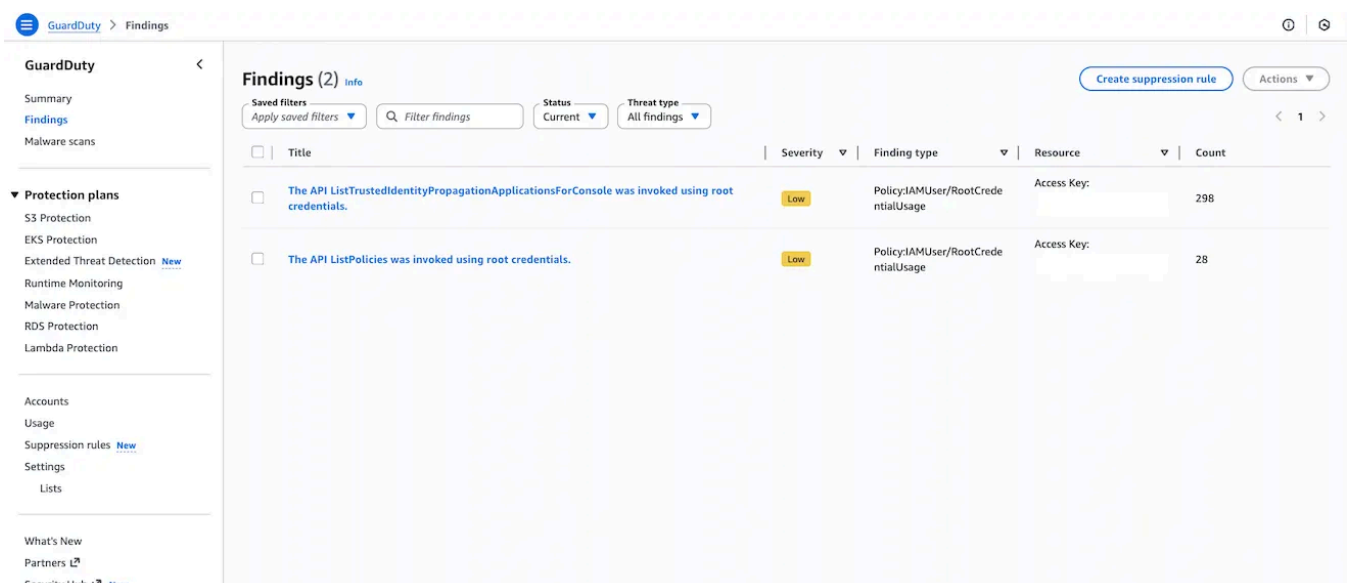
resource "aws_guardduty_detector" "primary" {
  enable                = true
  finding_publishing_frequency = "FIFTEEN_MINUTES"

  datasources {
    s3_logs {
      enable = true
    }
    kubernetes {
      audit_logs {
        enable = true
      }
    }
    malware_protection {
      scan_ec2_instance_with_findings {
        ebs_volumes {
          enable = true
        }
      }
    }
  }
}

resource "aws_guardduty_organization_admin_account" "delegated_admin" {
  count          = var.enable_delegated_admin ? 1 : 0
  admin_account_id = var.delegated_admin_account_id
}

```

Delegating GuardDuty administration to the security account means the security team can manage findings without needing access to the management account.



The screenshot shows the AWS GuardDuty console interface. On the left is a navigation sidebar with options like Summary, Findings, Malware scans, Protection plans, Accounts, and What's New. The main panel displays 'Findings (2)' with a table of detected issues. The table has columns for Title, Severity, Finding type, Resource, and Count. Two findings are listed, both with a 'Low' severity and related to 'The API ListTrustedIdentityPropagationApplicationsForConsole was invoked using root credentials.' and 'The API ListPolicies was invoked using root credentials.'.

Title	Severity	Finding type	Resource	Count
The API ListTrustedIdentityPropagationApplicationsForConsole was invoked using root credentials.	Low	Policy:IAMUser/RootCredentialUsage	Access Key: [REDACTED]	298
The API ListPolicies was invoked using root credentials.	Low	Policy:IAMUser/RootCredentialUsage	Access Key: [REDACTED]	28

Challenges and Lessons Learned

AWS Accounts Cannot Be Deleted

Once you create an AWS account through Organizations, you cannot delete it via Terraform. This caught me off guard. If you need to recover, your options are: remove the account from Terraform state using `terraform state rm`, move the account to the Suspended OU, and eventually close the account manually through the AWS Console.

IAM Identity Center Must Be Enabled Manually

IAM Identity Center cannot be enabled through Terraform. You have to go to the AWS Console and click Enable first. I handled this by wrapping the module in a conditional:

```
module "identity_center" {  
  source = "../../modules/identity-center"  
  count  = var.enable_identity_center ? 1 : 0  
}
```

This way, you deploy the Landing Zone in phases. First without Identity Center, enable it in the console, then set the variable to true and apply again.

Deployment Steps

Here is the order I followed to deploy the Landing Zone. The phased approach is important because some resources depend on others, and some require manual steps.

Phase 1: Foundation

Create a management account manually in AWS. Run `terraform init`. Apply with `create_accounts = false`. This creates the organization, OUs, and SCPs first.

Phase 2: Enable SSO

Go to AWS Console, navigate to IAM Identity Center, and click Enable. This step cannot be automated.

Phase 3: Create Accounts and Identity Center

Set `create_accounts = true` and `enable_identity_center = true`. Run `terraform apply`. This creates all 9 member accounts, sets up permission sets, groups, and account assignments.

Phase 4: Create Demo Users

Set `create_demo_users = true`. Run `terraform apply`. Then go to IAM Identity Center in the console and send password reset emails to each user.

Key Takeaways

- Start with design. Plan your OU structure before writing any code.
- Use modules. Keep Terraform code reusable and maintainable.
- Implement SCPs early. Guardrails prevent issues before they happen.
- Centralize security. Dedicated security accounts simplify auditing.
- Plan for the unexpected. AWS accounts cannot be deleted easily, so think before you create.

GitHub - lkravi/aws_landing_zone

Contribute to lkravi/aws_landing_zone development by creating an account on GitHub.

github.com

Resources

- AWS Organizations Documentation
docs.aws.amazon.com/organizations
- AWS Landing Zone Accelerator
aws.amazon.com/solutions/implementations/landing-zone-accelerator-on-aws
- Terraform AWS Provider
registry.terraform.io/providers/hashicorp/aws/latest
- AWS Well-Architected Framework
aws.amazon.com/architecture/well-architected

AWS

Aws Landing Zone

Organization

Terraform



Follow

Written by Ravi Yasakeerthi

206 followers · 50 following

www.linkedin.com/in/lkravi

